



DEADLOCKS (Ders 11) – TAM ÇALIŞMA & SINAV NOTU

Kaynak: [Ders11_Deadlocks.pdf](#) (78 slayt) – Dr. Öğr. Üyesi Ertan Bütün

İşaretler: ● = slaytta **kırmızı** (terim/madde) · ■ = **siyah koyu/bold** · 📄 = boşluk doldurma · 📊 = işlem/tablo sorusu · ⚠ = tuzak

Yapı: ① Kavramlar açık anlatım → ② Sınav notu → ③ Çıkabilecek sorular → ④ Çözümlü örnekler

① BÖLÜM: KAVRAMLARIN ANLAŞILIR ANLATIMI



Deadlock (Kilitlenme) Nedir?

Hayal et: İki kişi tek bir kapıdan geçmeye çalışıyor ama her ikisi de "önce sen geç" diyerek diğerinin hareket etmesini bekliyor. İkisi de sonsuza kadar bekler → **kimse geçemez**. İşte bu **deadlock**'tır.

● **Tanım:** Çoklu programlama (multiprogramming) ortamında, **bekleyen bir process'in talep ettiği kaynaklar, başka kaynakları bekleyen process'ler tarafından tutulduğu için bir daha asla durumunu değiştiremediği** duruma **deadlock** denir.

💡 **Önemli not:** İşletim sistemleri (Linux, Windows) genellikle deadlock önleme mekanizması sağlamaz → **program geliştiricinin** deadlock oluşmayacak şekilde kod yazması gerekir.



Sistem Modeli – Kaynak Kullanımı

Sistemde farklı türde **kaynaklar (resources)** vardır: CPU cycle, dosya, I/O cihazları (yazıcı, DVD). Bunlar $R_1, R_2, \dots, R_m$ ile gösterilir. Her kaynak türünün **bir veya birden fazla örneği (instance)** olabilir.

● Bir process kaynağı **3 adımda** kullanır:

- **Request (İstek):** Kaynak ister; uygun değilse **bekler**.
- **Use (Kullan):** Kaynak üzerinde işlemini yapar.
- **Release (Serbest bırak):** Kaynağı geri bırakır.

🔑 Bu işlemlerin gerçek karşılıkları: cihaz → `request()/release()`, dosya → `open()/close()`, hafıza → `allocate()/free()`, semafor → `wait()/signal()`, mutex → `acquire()/release()`.

★ Deadlock'ın 4 GEREKLİ KOŞULU (EN ÖNEMLİ KONU!)

Deadlock oluşması için aşağıdaki 4 koşulun AYNI ANDA gerçekleşmesi gerekir. Birini bile engellersen deadlock olmaz.

#	Koşul	Açık Anlatım
1	● Mutual Exclusion (Karşılıklı dışlama)	En az bir kaynak paylaşılamaz modda olmalı → bir anda sadece 1 process kullanabilir. (Örn: yazıcı)
2	● Hold and Wait (Tut ve bekle)	Bir process bir kaynağı tutarken , başka bir process'in tuttuğu başka kaynağı da bekliyor olmalı.
3	● No Preemption (Zorla alınamama)	Kaynak, tutan process'ten zorla alınamaz ; ancak process gönüllü olarak (iş bitince) bırakır.
4	● Circular Wait (Dairesel bekleme)	$\{P_0, P_1, \dots, P_n\}$ process'leri daireysel bekler: $P_0 \rightarrow P_1$ 'i, $P_1 \rightarrow P_2$ 'yi, ..., $P_n \rightarrow P_0$ 'i bekler.

⚠ **Çok kritik bağlantı:** Bu koşullar tamamen bağımsız değildir. **Circular wait** oluştuğunda, **hold and wait** da otomatik olarak vardır.

📊 Kaynak Tahsisi Grafi (Resource-Allocation Graph)

Deadlock'ı görselleştirmek için **yönlü graf (directed graph)** kullanılır.

- **Düğüm**ler (V): İki gruba ayrılır → **P** = process'ler (○ daire), **R** = kaynaklar (□ dikdörtgen).
- Kaynağın **birden fazla örneği** varsa → dikdörtgen içinde **her örnek bir nokta (•)** ile gösterilir.

● İki kenar (edge) türü:

- **Request edge (İstek kenarı):** $P_i \rightarrow R_j$ → P_i , R_j 'yi istiyor ve bekliyor.
- **Assignment edge (Atama kenarı):** $R_j \rightarrow P_i$ → R_j kaynağı P_i 'ye atanmış.

İstek karşılanınca: istek kenarı → atama kenarına dönüşür. Kaynak bırakılınca: atama kenarı silinir.

🔄 Döngü (Cycle) ile Deadlock İlişkisi — KRİTİK KURAL!

Durum	Sonuç
● Grafta döngü YOK	✅ Deadlock YOK (kesin)
● Döngü VAR + her kaynağın TEK örneği	❌ Deadlock VAR (döngü → gerekli VE yeterli koşul)
● Döngü VAR + kaynakların birden fazla örneği	⚠️ Deadlock OLABİLİR (döngü → gerekli ama YETERLİ DEĞİL)

🔧 Deadlock ile Başa Çıkma – 3 Genel Yol

● Deadlock problemine **3 farklı yaklaşım**:

- **Önleme/Kaçınma protokolü kullan** → sistem **hiçbir zaman** deadlock'a düşmez.
- **Deadlock'a izin ver, sonra algıla ve çöz.**
- **Problemi tamamen yok say** ("deadlock hiç olmayacak" gibi davran).

⚠️ **3. yaklaşım** işletim sistemlerinde **en yaygın** kullanılır (**Linux, Windows**) → deadlock yönetimini **uygulama geliştiriciye** bırakırlar.

İki ana yöntem grubu:

- **Deadlock Prevention (Önleme)**: Kaynak isteklerini **sınırlandırarak** 4 koşuldan birini engeller.
- **Deadlock Avoidance (Kaçınma)**: Her process'in **maksimum ihtiyacını önceden bildirmesini** ister, kaynak tahsis durumunu **dinamik** inceler.

🚫 DEADLOCK ÖNLEME (Prevention) – 4 Koşulu Tek Tek Kirmak

1 Mutual Exclusion'ı Kır

- Paylaşılabilir kaynaklar (örn. **read-only dosya**) zaten deadlock oluşturmaz.
- ⚠️ **AMA genelde işe yaramaz** → bazı kaynaklar **özünde paylaşılamaz** (örn. mutex lock aynı anda paylaşılamaz).

2 Hold and Wait'i Kır (2 protokol)

- ● **Protokol 1**: Process **çalışmaya başlamadan ÖNCE TÜM kaynaklarını** talep eder.
 - ⚠️ Dezavantaj: Kaynağı sonradan kullanacaksa bile **tüm süre boyunca tutar** → **verimsiz kullanım**.

- **Protokol 2:** Process **hiç kaynak tutmuyorken** yeni istek yapabilir. Yeni istek öncesi **elindekilerin hepsini bırakır**.
- ⚠ **Her iki protokolde de STARVATION mümkündür** (popüler kaynak bekleyen process süresiz bekleyebilir).

📌 **DVD örneği:** Process: DVD → disk kopyala, sırala, yazıcıya yazdır.

- Protokol 1: DVD + disk + yazıcıyı **baştan** ister (yazıcıyı sonda kullansa bile tutar = verimsiz).
- Protokol 2: Önce DVD+disk ister → kopyalar → bırakır → sonra disk+yazıcı ister.

3 No Preemption'ı Kır (2 protokol)

- **Protokol 1:** Process kaynak tutarken alamadığı yeni kaynak isterse → **elindeki tüm kaynaklar preempt edilir** (dolaylı serbest bırakılır), bekleme listesine eklenir. Eski + yeni kaynakları alabilince yeniden başlar.
- **Protokol 2:** İstenen kaynağı tutan process **başka kaynak bekliyorsa** → kaynak ondan alınıp (preempt) **yeni isteyene verilir**. Beklemiyorsa → **isteyen bekletilir**.

4 Circular Wait'i Kır (EN PRATİK YÖNTEM)

- Her kaynak türüne **farklı bir tam sayı (F fonksiyonu)** atanır → kaynaklar **sıralanır**.
- Process kaynak isteğini **yalnızca ARTAN sırada** yapabilir.
- P_i 'ye R_i atandıysa, ancak $F(R_j) > F(R_i)$ koşulunu sağlayan R_j 'yi isteyebilir.
- **İspat:** Eğer circular wait olsaydı → $F(R_0) < F(R_1) < \dots < F(R_n) < F(R_0)$ çelişkisi çıkar. $F(R_n) < F(R_0)$ olamayacağı için circular wait OLUŞAMAZ.

⚠ DEADLOCK'TAN KAÇINMA (Avoidance)

● **Temel fikir:** Her process **maksimum kaynak ihtiyacını önceden bildirir**. Sistem, kaynak tahsis durumunu **dinamik** inceleyerek **circular wait'in asla oluşmayacağından** emin olur.

⚠ **Önleme yöntemleri cihazlardan az faydalanmaya ve düşük throughput'a neden olur; kaçınma bu yüzden alternatiftir.**

🔑 Safe State (Güvenli Durum) – ÇOK ÖNEMLİ KAVRAM

● **Safe State:** Sistem, kaynakları process'lere **belirli bir sırayla (safe sequence)** maksimum ihtiyaçları kadar atayıp **deadlock olmadan** hepsini tamamlayabiliyorsa, sistem **güvenli** durumdadır.

● **Safe Sequence** $\langle P_1, P_2, \dots, P_n \rangle$: Her P_i 'nin ihtiyacı, **mevcut boş kaynaklar + ondan önceki tüm P_j 'lerin (j ile karşılanabiliyorsa geçerlidir. P_i tamamlanınca kaynaklarını iade eder, sıradaki P_{i+1} çalışır...**

● Temel kurallar:

Durum	Sonuç
Sistem safe state	✅ Deadlock YOK
Sistem unsafe state	⚠️ Deadlock OLMA İHTİMALİ VAR

💡 Görsel mantık: $deadlock \subset unsafe \subset tüm\ durumlar$. Deadlock'tan kaçınma = sistemin asla unsafe state'e girmemesini sağlamak. (Unsafe = OS artık deadlock'ı engelleyemeyebilir.)

🔺 Kaynak Tahsisi Graf Algoritması (TEK örnekli kaynaklar için)

- İstek/atama kenarlarına ek olarak claim edge (niyet kenarı) eklenir → $P_i \dashrightarrow R_j$ (noktalı çizgi): "Pi ileride Rj'yi isteyebilir."
- Process çalışmaya başlamadan tüm niyet kenarları grafta olmalı.
- Kural: P_i, R_j 'yi isterse → istek kenarını atama kenarına çevirmek döngü oluşturmuyorsa izin verilir (safe). Döngü oluşuyorsa → unsafe → P_i bekler.

🏦 Banker's Algorithm (Banker Algoritması) – ÇOK örnekli kaynaklar için

● Graf algoritması çok örnekli kaynaklarda çalışmaz → Banker algoritması kullanılır.

💡 İsmi'nin mantığı: Banka, parasını hiçbir zaman tüm müşterilerin ihtiyacını karşılayamayacak şekilde dağıtmaz. Yeni process maksimum ihtiyacını bildirir; istek ancak sistemi safe state'te bırakacaksa kabul edilir.

● 4 veri yapısı (n =process sayısı, m =kaynak türü sayısı):

Yapı	Boyut	Anlamı
● Available	m vektör	Her kaynaktan kaç boş örnek var
● Max	$n \times m$ matris	Her process'in maksimum talebi
● Allocation	$n \times m$ matris	Her process'in şu an kullandığı kaynak
● Need	$n \times m$ matris	Kalan ihtiyaç = $\blacksquare$ Need = Max – Allocation

🔍 İki Alt Algoritma:

A) Safety Algorithm (Güvenlik Algoritması) – sistem güvenli mi?

1. $Work = Available$; $Finish[i] = false$ (tüm i)
2. Şu koşulu sağlayan i bul: $Finish[i] == false$ VE $Need_i \leq Work$
→ varsa adım 3'e, yoksa adım 4'e

3. `Work = Work + Allocation_i ; Finish[i] = true ;` → adım 2'ye dön
4. Tüm `Finish[i]==true` ise → SİSTEM GÜVENLİ

B) Resource-Request Algorithm – P_i'nin Request isteği onaylanır mı?

1. `Request_i ≤ Need_i` ? değilse → HATA (maksimumu aştı)
2. `Request_i ≤ Available` ? değilse → P_i BEKLER (kaynak yok)
3. Geçici tahsis yap:
`Available = Available - Request_i`
`Allocation_i = Allocation_i + Request_i`
`Need_i = Need_i - Request_i`
→ Güvenlik algoritmasını çalıştır:
Güvenli ise → tahsis et; Değilse → P_i bekler, eski duruma dön

DEADLOCK ALGILAMA (Detection)

Sistem önleme/kaçınma kullanmıyorsa deadlock olabilir → algılamak için iki durum:

A) Her kaynaktan TEK örnek → Wait-For Graph

-  Wait-for grafi: Kaynak tahsisi grafindan kaynak düğümleri çıkarılır, sadece process'ler kalır.
-  `Pi → Pj` kenarı: "P_i, P_j'yi bekliyor."
-  Wait-for grafinda DÖNGÜ varsa → DEADLOCK var.
- Periyodik olarak döngü arayan algoritma çalıştırılır.

B) Her kaynaktan ÇOK örnek → Banker'a benzer algoritma

 Veri yapıları: Available, Allocation, Request (Request_i=k → P_i, R_j'den k tane daha istiyor).

1. `Work = Available`
`Allocation_i ≠ 0` ise `Finish[i]=false`, değilse `Finish[i]=true`
2. `Finish[i]==false` VE `Request_i ≤ Work` olan i bul → varsa 3, yoksa 4
3. `Work = Work + Allocation_i ; Finish[i]=true ;` → 2'ye dön
4. BAZI `Finish[i]==false` ise → SİSTEM DEADLOCK'TA

 **Safety vs Detection farkı:** Safety'de Need kullanılır; Detection'da Request kullanılır. Detection'da Allocation=0 olanlar baştan Finish=true.

Algılama Algoritması NE ZAMAN Çalışmalı?

- İki faktöre bağlı: (1) Deadlock ne sıklıkta olur? (2) Kaç process etkilenir?
 - Sık deadlock → sık çalıştır.
 - ⚠ Her istekte çalıştırmak pahalı (yüksek ek yük).
 - ● Ucuz alternatif: belirli aralıklarla çalıştır (örn. saatte bir, veya CPU kullanımı %40'ın altına düşünce).
-

DEADLOCK'TAN KURTULMA (Recovery)

- Deadlock algılanınca: ya operatöre bildir (manuel), ya otomatik kurtul. İki seçenek:

Seçenek 1: Process Sonlandırma (Abort)

- İki yöntem:
 1. ● Tüm deadlock process'lerini sonlandır → sonuçlar kaybolur (pahalı ama kesin).
 2. ● Döngü kırılana kadar birer birer sonlandır → her sonlandırmada döngü kontrolü yapılır.
- ⚠ Abort riski: Process dosya güncellemenin/mutex tutmanın ortasındaysa → veri tutarsızlığı oluşabilir.
- Hangi process sonlandırılmalı? = ekonomik/maliyet sorunu. Etkileyen 6 faktör:
 1. Önceliği · 2. Ne kadar çalıştığı / kalan süresi · 3. Kullandığı kaynak sayısı-türü · 4. Tamamlanması için gereken kaynak · 5. Kaç process sonlanmalı · 6. Etkileşimli (interactive) mi, toplu (batch) mu

Seçenek 2: Kaynak Preemption (Zorla alma)

- Döngü kırılana kadar kaynaklar sırayla alınıp başka process'e verilir. 3 konu:
 1. ● Selecting a victim (Kurban seçimi): Maliyeti en aza indirecek sıra.
 2. ● Rollback: Kaynağı alınan process güvenli duruma geri alınır, oradan yeniden başlar.
 3. ● Starvation: Hep aynı process kurban seçilirse açlık olur → maliyet faktörüne sonlandırma sayısını da kat (önlem).
-

② BÖLÜM: SINAVA HAZIRLIK – ÖZET KARTLARI



Tanım Hızlı Tekrar

Terim	Tek Cümle
Deadlock	Process'lerin karşılıklı, sonsuz bekleyip ilerleyemediği durum
Mutual Exclusion	Kaynak paylaşılamaz, bir anda 1 process
Hold and Wait	Kaynak tutarken başkasını beklemek
No Preemption	Kaynak zorla alınamaz
Circular Wait	$P_0 \rightarrow P_1 \rightarrow \dots \rightarrow P_n \rightarrow P_0$ dairesel bekleme
Request/Assignment edge	$P_i \rightarrow R_j$ istek / $R_j \rightarrow P_i$ atama
Claim edge	$P_i \dashrightarrow R_j$ niyet (noktalı, ileride isteyebilir)
Wait-for graph	Kaynaksız, sadece process'li graf; döngü=deadlock
Safe state	Safe sequence ile deadlocksuz tamamlanabilen durum
Need	$Max - Allocation$
Banker's algorithm	Çok örnekli kaynakta avoidance



Ezberlenecek KESİN Kurallar

- Döngü yok $\rightarrow$ deadlock yok (her zaman)
- Döngü + tek örnek $\rightarrow$ deadlock VAR (gerekli+yeterli)
- Döngü + çok örnek $\rightarrow$ deadlock OLABİLİR (gerekli, yeterli değil)
- Safe $\rightarrow$ deadlock yok • Unsafe $\rightarrow$ deadlock olabilir
- Need = Max – Allocation
- 4 koşulun HEPSİ aynı anda olmalı; circular wait $\rightarrow$ hold-and-wait içerir
- Safety algoritması Need kullanır, Detection Request kullanır
- Linux & Windows $\rightarrow$ 3. yöntem (yok say), yönetimi geliştiriciye bırakır



En Çok Karıştırılanlar

- Prevention (4 koşuldan birini kır) $\leftrightarrow$ Avoidance (max ihtiyaç bilinir, dinamik kontrol)
- Avoidance (deadlock oluşmadan engelle) $\leftrightarrow$ Detection (oluşmasına izin ver, sonra bul)
- Graf algoritması (tek örnek) $\leftrightarrow$ Banker (çok örnek)
- Banker Need $\leftrightarrow$ Detection Request
- Recovery: Abort (process öldür) $\leftrightarrow$ Preemption (kaynak al)

③ BÖLÜM: ÇIKABİLECEK SORULAR

Tanım/Kavram Soruları

1. Deadlock'ın 4 gerekli koşulunu yazıp kısaca açıklayınız. (klasik soru!)
2. Circular wait oluştuğunda hangi koşul da otomatik var olur? (Hold and wait)
3. Request edge ile assignment edge arasındaki fark nedir?
4. Kaynak tahsisi grafında döngü varsa kesinlikle deadlock var mıdır? Hangi durumda evet, hangisinde belki?
5. Safe state, unsafe state ve deadlock arasındaki ilişkiyi açıklayın. (deadlock $\subset$ unsafe)
6. Deadlock prevention ile avoidance arasındaki fark nedir?
7. Wait-for grafi nasıl elde edilir, deadlock'ı nasıl gösterir?
8. Deadlock algılama algoritması ne zaman çalıştırılmalı?
9. Deadlock'tan kurtulmanın iki yolu nedir? (abort / preemption)
10. Kurban seçiminde (victim) starvation neden oluşur, nasıl önlenir?

Boşluk Doldurma (📝)

- Bir anda sadece bir process'in kaynağı kullanması (mutual exclusion).
- $Need = \text{_____} - \text{_____}$ (Max – Allocation).
- Tek örnekli kaynaklarda deadlock algılama için grafi kullanılır (wait-for).
- Sistem state'te ise deadlock yoktur (safe).
- Çok örnekli kaynakta kaçınma için algoritması kullanılır (Banker's).
- İleride istenebilecek kaynağı gösteren kenar (claim edge / niyet kenarı).

İşlem Soruları (🧩) — kesin gelir!

- Bir kaynak tahsisi grafi verilip "deadlock var mı?" sorusu.
- Banker's algorithm: Need matrisini hesaplayıp safe sequence bulma.
- Resource-Request: bir Request'in onaylanıp onaylanmayacağı (safe mi?).
- Detection: verilen Allocation/Request tablosunda deadlock var mı?

④ BÖLÜM: ÇÖZÜMLÜ ÖRNEKLER



ÖRNEK 1: Kaynak Tahsisi Grafi – Deadlock var mı?

Durum: $R_1(1 \text{ örnek}), R_2(2), R_3(1), R_4(3)$. P_1 R_2 'yi tutar R_1 'i bekler; P_2 R_1 ve R_2 tutar R_3 bekler; P_3 R_3 tutar. → Bu durumda döngü yok → DEADLOCK YOK.

Şimdi P₃, R₂'den bir örnek istesin. İki döngü oluşur:

P₁ → R₁ → P₂ → R₃ → P₃ → R₂ → P₁
P₂ → R₃ → P₃ → R₂ → P₂

✓ Sonuç: P₁, P₂, P₃ DEADLOCK olur. (P₂, P₃'ün R₃'ünü bekler; P₃, R₂'yi bekler; P₁, P₂'nin R₁'ini bekler.)

⚠ Karşı örnek: $P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$ döngüsü olsa bile, eğer P₄, R₂'yi bırakırsa P₃ kullanabilir → döngü kırılır → DEADLOCK YOK. (Çok örnekli kaynakta döngü ≠ kesin deadlock.)

🎲 ÖRNEK 2: BANKER'S ALGORITHM – Safe Sequence Bulma (en önemli örnek!)

Sistem: 5 process (P₀–P₄), 3 kaynak: A(10), B(5), C(7). To anı:

Process	Allocation	Max	Need = Max–Alloc
P ₀	0 1 0	7 5 3	7 4 3
P ₁	2 0 0	3 2 2	1 2 2
P ₂	3 0 2	9 0 2	6 0 0
P ₃	2 1 1	2 2 2	0 1 1
P ₄	0 0 2	4 3 3	4 3 1

Available (boş) = (3 3 2) → bu **Work** 'ün başlangıcı.

Adım adım Safety Algorithm:

Adım	Work	Kontrol	Sonuç
Başla	3 3 2	P ₀ Need(7 4 3) ≤ (3 3 2)? HAYIR	P ₀ atla
	3 3 2	P ₁ Need(1 2 2) ≤ (3 3 2)? EVET	✓ P ₁ seç, Work += Alloc(2 0 0)
	5 3 2	P ₃ Need(0 1 1) ≤ (5 3 2)? EVET	✓ P ₃ seç, Work += (2 1 1)
	7 4 3	P ₄ Need(4 3 1) ≤ (7 4 3)? EVET	✓ P ₄ seç, Work += (0 0 2)
	7 4 5	P ₀ Need(7 4 3) ≤ (7 4 5)? EVET	✓ P ₀ seç, Work += (0 1 0)
	7 5 5	P ₂ Need(6 0 0) ≤ (7 5 5)? EVET	✓ P ₂ seç, Work += (3 0 2)
Bitti	10 5 7	Tüm Finish=true	🏆 GÜVENLİ

✓ SAFE SEQUENCE = <P₁, P₃, P₄, P₀, P₂> → Sistem güvenli durumda.

💡 **Çözüm ipucu:** Her adımda $Need \leq Work$ olan ilk process'i seç, onun Allocation'ını Work'e ekle. Hepsi bitince güvenli.

🎲 ÖRNEK 3: RESOURCE-REQUEST – İstek onaylanır mı?

Soru: P_1 , $Request_1 = (1, 0, 2)$ istiyor. ($Available=3\ 3\ 2$, $Need_1=1\ 2\ 2$)

Adım 1: $Request_1 \leq Need_1 \rightarrow (1,0,2) \leq (1,2,2)$? ☒ true Adım 2: $Request_1 \leq Available \rightarrow (1,0,2) \leq (3,3,2)$? ☒ true Adım 3: Geçici tahsis yapılır:

- $Available = (3,3,2) - (1,0,2) = (2, 3, 0)$
- $Allocation_1 = (2,0,0) + (1,0,2) = (3,0,2)$ • $Need_1 = (0,2,0)$
- Güvenlik algoritması $\rightarrow \langle P_1, P_3, P_4, P_0, P_2 \rangle$ safe sequence bulunur.

☒ **Sonuç:** Sistem güvenli $\rightarrow$ İstek DERHAL KABUL EDİLİR.

🎲 ÖRNEK 4: REDDEDİLEN İstek

Soru: (Örnek 3 sonrası) P_4 , $Request_2 = (3, 3, 0)$ istiyor. ($Available$ şimdi $=2\ 3\ 0$, $Need_4=4\ 3\ 1$)

Adım 1: $(3,3,0) \leq Need_4(4,3,1)$? ☒ true Adım 2: $(3,3,0) \leq Available(2,3,0)$? ☒ FALSE (A'da 3 isteniyor ama 2 var)

☒ **Sonuç:** Adım 2 sağlanmadı $\rightarrow$ kaynaklar yetersiz $\rightarrow$ İstek REDDEDİLİR (P_4 bekler).

🎲 ÖRNEK 5: KAYNAKLAR VAR AMA UNSAFE $\rightarrow$ Reddedilir

Soru: P_0 , $Request_3 = (0, 2, 0)$ istiyor. ($Available=2\ 3\ 0$)

- Adım 1 ve 2 geçer (kaynaklar kullanılabilir), ama geçici tahsis sonrası güvenlik algoritması safe sequence BULAMAZ $\rightarrow$ unsafe.

☒ **Sonuç:** Kaynaklar yeterli olsa bile yeni durum UNSAFE $\rightarrow$ İstek KABUL EDİLMEZ.

⚠️ **Önemli ders:** Kaynak var olması yeterli değil; tahsis sonrası safe state korunmalı!



ÖRNEK 6: DEADLOCK DETECTION – Deadlock var mı?

Sistem: 5 process, kaynaklar A(7), B(2), C(6), To anı:

Process	Allocation	Request	
P ₀	0 1 0	0 0 0	
P ₁	2 0 0	2 0 2	
P ₂	3 0 3	0 0 0	
P ₃	2 1 1	1 0 0	
P ₄	0 0 2	0 0 2	

Available = (0 0 0)

Detection çalıştır: Request ≤ Work olanları sırayla bul:

- Work=(0 0 0): P₀ Request(0 0 0) ≤ (0 0 0) ✓ → Work+=Alloc(0 1 0)=(0 1 0)
- P₂ Request(0 0 0) ≤ (0 1 0) ✓ → Work+=(3 0 3)=(3 1 3)
- P₃ Request(1 0 0) ≤ (3 1 3) ✓ → Work+=(2 1 1)=(5 2 4)
- P₄ Request(0 0 2) ≤ (5 2 4) ✓ → Work+=(0 0 2)=(5 2 6)
- P₁ Request(2 0 2) ≤ (5 2 6) ✓ → Work+=(2 0 0)=(7 2 6)

✓ Tüm Finish=true → <P₀, P₂, P₃, P₄, P₁> → DEADLOCK YOK.



Şimdi T₁'de P₂, (0, 0, 1) istesin → Request₂ = (0 0 1):

- Work=(0 0 0): Sadece P₀ Request(0 0 0) ≤ (0 0 0) ✓ → Work=(0 1 0)
- Diğer hiçbir Request ≤ Work sağlamaz! (P₁ ister 2 0 2, P₂ ister 0 0 1 ama C'de 0 var, P₃ ister 1 0 0, P₄ ister 0 0 2...)

✗ P₀ hariç hepsinin Finish=false → P₁, P₂, P₃, P₄ DEADLOCK olur!



Sonuç: Tek bir küçük istek (P₂'nin 1 tane C istemesi) bile sistemi deadlock'a sokabilir.



Başarılar! En sık gelenler: 4 koşul (tanım+açıklama), döngü-deadlock ilişkisi, Banker safe sequence hesabı ve Resource-Request kabul/red analizi. İşlem sorularında mutlaka Need = Max – Allocation ile başla ve her adımda Need(veya Request) ≤ Work kontrolünü tablo halinde yaz.